STATE MIND

Lido CSM V3 and CM V2

30-03-2026 - 30-06-2026

Table of contents

1. Project brief

Title	Description
Client	Lido
Project name	Lido CSM V3 and CM V2
Timeline	30-03-2026 – 30-06-2026

Project Log

Repository	https://github.com/lidofinance/community-staking-module	
Date	**Commit Hash**	**Note**
06-04-2026	c8cf8833a4c50d69e2e20271d3282a1d99f6948e	Initial Commit
15-06-2026	403c2bd7b6a8f94ade203e610cfd0e992b55c5c0	Reaudit commit
30-06-2026	4d3de6658499e1c1774951780a97d7ae25ca18b8	Reaudit commit 2

Short Overview

Community Staking Module V3 is a technical upgrade to the existing CSM v2. It introduces native support for 0x02 validator withdrawal credentials under a separate module instance, built-in Node Operator reward splitters, improved reward-claim and penalty-processing flows, and more granular role-based access control for managing Node Operator type parameters.

Curated Module V2 is the upgrade and new version of the Lido Curated Module and is based on the CSM codebase. It is designed to streamline Node Operator operations, simplify onboarding, provide dedicated treatment for public-good Node Operators, increase Lido protocol security, and improve stake distribution with an emphasis on decentralization and Node Operator diversity.

The main features introduced in the audited scope include:

Common CSM v3 and CM v2 features:

- Support for 0x02 validator withdrawal credentials, including validator deposits, withdrawals, updated accounting for validators with different balances, and support for incoming and outgoing validator consolidations.
- Built-in Node Operator reward splitters, allowing Node Operator rewards to be separated from bond-related claims and improving reward distribution flexibility.
- Refined penalty handling, where delayed penalties may be reported on-chain, uncovered penalty amounts are recorded as bond debt, and Node Operator limits may be adjusted when penalties exceed the available bond.
- Improved reward-claim logic and bond accounting, including changes required to support the updated validator lifecycle and penalty model.
- More granular role-based access control for managing Node Operator type parameters and related module configuration.

CM V2–specific features:

- Weight–based stake allocation, enabling stake to be distributed between Curated Node Operators according to configured allocation weights.
- Meta Operators Registry support, allowing Node Operators to manage sub–node operators while preserving fair stake allocation and accounting for legacy CM v1 stake during migration.
- Extended Node Operator address management, including permissioned mechanisms for the Lido DAO to forcefully update Node Operator addresses in exceptional cases.
- Permissioned Node Operator creation without immediate validator key upload or bond provision, reflecting the controlled onboarding model of the Curated Module.

Project Scope

The audit covered the following files:

- BaseModule.sol
- Accounting.sol
- NodeOperatorOps.sol
- CSModule.sol
- BondCore.sol
- SigningKeys.sol
- WithdrawnValidatorLib.sol
- BondCurvesLib.sol
- BondLock.sol
- FeeSplits.sol
- DepositAllocatorGreedy.sol
- BondCurve.sol
- GIndex.sol
- PackedSortKeyMaxHeapLib.sol
- Types.sol
- NamedUpgradeable.sol
- ValidatorCountsReport.sol
- PausableWithRoles.sol
- ValidatorBalanceLimits.sol
- HashConsensus.sol
- CuratedDepositAllocator.sol
- Verifier.sol
- CuratedModule.sol
- FeeDistributor.sol
- StakeTracker.sol
- NOAddresses.sol
- VettedGate.sol
- PermissionlessGate.sol
- FeeOracle.sol
- TopUpQueueOps.sol
- CuratedGate.sol
- OssifiableProxy.sol
- ExternalOperatorLib.sol
- MerkleGateFactory.sol
- UnstructuredStorage.sol
- PackedSortKeyLib.sol
- KeyPointerLib.sol
- ParametersRegistry.sol
- MetaRegistry.sol
- DepositQueueOps.sol
- BaseOracle.sol
- SSZ.sol
- ValidatorStrikes.sol
- Ejector.sol
- ExitPenalties.sol
- DepositQueueLib.sol
- MerkleGate.sol
- TopUpQueueLib.sol
- GeneralPenaltyLib.sol
- AssetRecovererLib.sol
- TransientUintUintMapLib.sol
- ModuleLinearStorage.sol
- AssetRecoverer.sol
- OperatorTracker.sol
- ExitTypes.sol

2. Finding severity breakdown

All vulnerabilities discovered during the audit are classified based on their potential severity and have the following classification:

Severity	Description
Critical	Bugs leading to assets theft, fund access locking, or any other loss of funds to be transferred to any party.
High	Bugs that can trigger a contract failure. Further recovery is possible only by manual modification of the contract state or replacement.
Medium	Bugs that can break the intended contract logic or expose it to DoS attacks, but do not cause direct loss of funds.
Informational	Bugs that do not have a significant immediate impact and could be easily fixed.

Based on the feedback received from the Client regarding the list of findings discovered by the Contractor, they are assigned the following statuses:

Status	Description
Fixed	Recommended fixes have been made to the project code and no longer affect its security.
Acknowledged	The Client is aware of the finding. Recommendations for the finding are planned to be resolved in the future.

3. Summary of findings

Severity	# of Findings
Critical	0 (0 fixed, 0 acknowledged)
High	0 (0 fixed, 0 acknowledged)
Medium	0 (0 fixed, 0 acknowledged)
Informational	20 (14 fixed, 6 acknowledged)
Total	20 (14 fixed, 6 acknowledged)

4. Conclusion

During the audit of the codebase, 20 issues were found in total:

- 20 informational severity issues (14 fixed, 6 acknowledged)

The final reviewed commit is 4d3de6658499e1c1774951780a97d7ae25ca18b8

Deployment

All deployments were successfully validated. The deployed bytecode of each contract within the audited scope matches the bytecode produced from the final audited commit: 4d3de6658499e1c1774951780a97d7ae25ca18b8.

CSM V3 scope

Contract	Address
Accounting (Implementation)	0xe768572cc5aE5C698345C59288d871a949Ea8bd3
CSModule (Implementation)	0x63992a86f009fcC796a8369feEfB68880aef4e3a
Ejector	0x610B517D380f287c239C93F8eF6FfBd567AA4bA5

ExitPenalties (Implementation)	0xA5b9e96E951089E629Ab0834AEaF242a81394EA0
FeeDistributor (Implementation)	0x936da7cDB7eed1084d294E23eA1d7Ad72DCcfE0E
FeeOracle (Implementation)	0xecE6e0Cde61078F76b66Ef0C338a6875E5D01F79
IdentifiedDVTClusterGate (Proxy)	0xa12760721A72A7199aB38059DA6690b9Cd4ed7B8
ParametersRegistry (Implementation)	0x107d287F178cD54792614d7D63C47D8242240BeD
PermissionlessGate	0xb8cd8F059Ad7a5dB8CAfDe34aAb007317F7156C8
ValidatorStrikes (Implementation)	0xd25E7C3923d2e68c325980b0e15eD20d62B2691F
Verifier	0xfce7aB839e55de77730716D05b3553e45ab3A5Ba
VettedGateFactory (MerkleGateFactory)	0xc0f110Af6eA9119037a71C84D14506A22AE43DdE
VettedGate (Implementation)	0x66ADb8b3F58d3DFdF6bAdB595E41f19e947E5c14
DepositQueueOps	0xb430AA6C70A352c2aaC9813AE049A210dB11aB41
TopUpQueueOps	0xdA104f5f2a18405fC7cCD6E0A7FEB5B824843606

CM V2 scope

Contract	Address
Accounting (Proxy)	0x2F91e3A8C5d6593bf4F8403fCfeCcd62dF59f6F6
Accounting (Implementation)	0xB41F5d2721906b3BE4fC7ae08261266C801076C2
CuratedGateFactory (MerkleGateFactory)	0xDdE99d63b352A665d04339D4792E6852Ce89d1B7
CuratedGate (Implementation)	0x3cb948FD454ad6b20DE67633f25DcbDbEaa0e849
CuratedGate (Proxy)	0x6093EFA6B5E2FF3be54d1c895c9deA932805c49F
CuratedGate (Proxy)	0x8c002c6eE10cf8adb78D1F9EB2e134FdaF8A7C1a
CuratedGate (Proxy)	0x207798e6fD1aa7Ee8a63782A64c959cD6727b78C
CuratedGate (Proxy)	0xeF273Ca4A21Ba7B414Ae3C9f9b443038cb133F72
CuratedGate (Proxy)	0x3BbBb175f7F07954DE00052b20E1c5572223F24D
CuratedGate (Proxy)	0x86A8d4E0db5938D21d98047544668FCCB1A9ADc8

CuratedGate (Proxy)	0x773933F9db8964A17d62fb808f2EC7A2de4247CC
CuratedModule (Proxy)	0xDa5F930cE326EB5205085D66c72A4E79d60cB8C1
CuratedModule (Implementation)	0x959fC67FE53c8A6C7a1AEd73430Aa07a36eD9337
Ejector	0xe181A377A2d2BDE9A83f1474BC3DB7A412de091E
ExitPenalties (Proxy)	0x004aFb7DAA7dEA20EbAaB75c9F4892C879FaCCe0
ExitPenalties (Implementation)	0x3766ABbA4635EE0fC3E6A7EB3EF169fe930df9c2
FeeDistributor (Proxy)	0x367d23c756599c20DCc8D6943F4976E8F88D60d7
FeeDistributor (Implementation)	0x7C8FEE1dcC95Df60fC9b5BE7603c28Eb3af16753
FeeOracle (Proxy)	0x8EeFCdbD984c30E472BcbF545783D051CB5114e5
FeeOracle (Implementation)	0x16804084408B6Caad10046F49aD421cBA56C0b5e
HashConsensus	0x902D64c93F6595339aA46105627a085591051aFb
MetaRegistry (Proxy)	0xA64b339eebD3dC3De848298B6a140955932901d8
MetaRegistry (Implementation)	0x6d852907463496622bb5FE5bc55cc30C4682E10e
ParametersRegistry (Proxy)	0xffC1C5d59CeAC6F6c27E701F04a70cb50474607C
ParametersRegistry (Implementation)	0xfF419BbBC5f44d46547079922a88d691886d192a
ValidatorStrikes (Proxy)	0xf4618370a1fBf46905B16C10817c8CFaD924D6db
ValidatorStrikes (Implementation)	0x239Ee6ab18fB06370ad53Ce05097B32208fB0a30
Verifier	0xC392F457960f1B13Ebaf1aa6C065479dD507E1E3
CuratedDepositAllocator	0xa4fCD4dDa0e4a847142E3592C97c77d8B9B3Cf5F

Common external libraries

Contract	Address
AssetRecovererLib	0x37aDa408AE3c3992953688e2CCb9eE7a3dfdA902
BondCurvesLib	0xC4511d09639e5E174506083443da230D39196323
GeneralPenalty	0xF05545ED71c60bBba6E73B6B70B15D4f5F22C0f4

NOAddresses	0x9D9c8799189c797f6e2dA74F71aDF84492adA7D3
NodeOperatorOps	0xDD42EE5D54A1822021782F3F455bb99fBC19499A
StakeTracker	0xbb6E4Db18182d45038F91B9F1195291c206fd8d2
WithdrawnValidatorLib	0x3bf9674f062aF9BA94FdAe9Fcdf2D0001FFf0a3A

5. Findings report

INFORMATIONAL-01	Multiple interface errors are not used	Fixed at: e57c3a1

Description

Lines:

- ICuratedGate.sol#L15
- IAccounting.sol#L43-L44
- ICuratedModule.sol#L13-L14
- IExitPenalties.sol#L24
- IValidatorStrikes.sol#L23-L24
- IMetaRegistry.sol#L40
- IMetaRegistry.sol#L42
- IMetaRegistry.sol#L49
- IMetaRegistry.sol#L50
- IMetaRegistry.sol#L53

Several interface-level error declarations are currently dead declarations: they are not referenced via their own interface prefix anywhere in the codebase. In practice, these errors are either duplicated in other interfaces and used from there or not used at all.

Recommendation

We recommend removing the unused error declarations that are not referenced in the codebase and keeping only the actively used ones.

INFORMATIONAL-02	Deprecated referrer parameter is still accepted and emitted	Acknowledged

Description

Lines:

- BaseModule.sol#L84
- CSModule.sol#L82-L90
- PermissionlessGate.sol#L40-L45
- PermissionlessGate.sol#L64-L69
- PermissionlessGate.sol#L89-L94
- VettedGate.sol#L47-L54
- VettedGate.sol#L74-L81
- VettedGate.sol#L102-L109

The **referrer** parameter in **createNodeOperator** is deprecated – **BaseModule** ignores it (**address /* referrer */**). However, **CSModule** still emits **ReferrerSet**, and all gates (**PermissionlessGate**, **VettedGate**) propagate the value through the entire call chain. This creates a misleading interface: callers supply a value with no on-chain effect beyond an event.

Recommendation

We recommend removing the **referrer** parameter, **ReferrerSet** event, and all related propagation from gates and interfaces. If the event is intentionally kept for off-chain indexing, document it explicitly as informational-only.

Client's comments

The **referrer** argument had multiple uses in the previous version of CSM. For now, it is still necessary to emit an event that allows CSM maintainers to determine which integration the Node Operator was created in.

INFORMATIONAL-03	Redundant storage writes and events on unchanged parameter updates	Acknowledged

Description

Lines:

- ParametersRegistry.sol
- StakeTracker.sol#L147-L164

1. ParametersRegistry setter functions unconditionally write to storage and emit events on each call, even when the new value equals to the currently stored value. This behavior can generate unnecessary events and governance noise.
2. **StakeTracker._increaseKeyAllocatedBalance()** writes and emits an event even when the resulting value is unchanged. This can add avoidable gas and noisy telemetry for top-up flows.

Recommendation

1. We recommend adding same-value guards to setter functions where equality checks are cheap (e.g., scalar and small struct values). For example:

```
function setKeyRemovalCharge(
    uint256 curveId,
    uint256 keyRemovalCharge
) external onlyRoleMemberOrCurveParametersRoleOrAdmin(MANAGE_GENERAL_PENALTIES_AND_CHARGES_ROLE) {
    MarkedUint248 storage current = _keyRemovalCharges[curveId];
    if (current.isValue && current.value == keyRemovalCharge) return;
    _keyRemovalCharges[curveId] = MarkedUint248(keyRemovalCharge.toUint248(), true);
    emit KeyRemovalChargeSet(curveId, keyRemovalCharge);
}
```

2. We recommend adding a no-op fast path similar to **StakeTracker._setOperatorBalance()**.

Client's comments

StakeTracker._increaseKeyAllocatedBalance() has already been fixed in - https://github.com/lidofinance/community-staking-module/commit/f37354a6248ce2568f22e7792c447ba24813b755
Setters in **ParametersRegistry.sol** are expected to be called by privileged actors only in rare cases. Hence, we prefer to keep them intact.

INFORMATIONAL-04	Inline role check without dedicated wrapper function	Acknowledged

Description

Lines:

- BaseModule.sol#L286
- BaseModule.sol#L346
- BaseModule.sol#L352
- CSModule.sol#L169
- CSModule.sol#L189

Several external functions in **BaseModule** and **CSModule** call **_checkRole()** directly rather than through a dedicated internal wrapper. This is inconsistent with the established pattern used for **STAKING_ROUTER_ROLE**, **REPORT_GENERAL_DELAYED_PENALTY_ROLE**, **VERIFIER_ROLE**, and **CREATE_NODE_OPERATOR_ROLE**, each of which has a named wrapper.

Recommendation

We recommend introducing named wrapper functions for the five remaining roles, consistent with the pattern established for **_checkStakingRouterRole()** and peers. In **BaseModule**:

```
function _checkSettleGeneralDelayedPenaltyRole() internal view {
    _checkRole(SETTLE_GENERAL_DELAYED_PENALTY_ROLE);
}

function _checkReportSlashedWithdrawnValidatorsRole() internal view {
    _checkRole(REPORT_SLASHED_WITHDRAWN_VALIDATORS_ROLE);
}

function _checkReportRegularWithdrawnValidatorsRole() internal view {
    _checkRole(REPORT_REGULAR_WITHDRAWN_VALIDATORS_ROLE);
}
```

And in **CSModule**:

```
function _checkManageTopUpQueueRole() internal view {
    _checkRole(MANAGE_TOP_UP_QUEUE_ROLE);
}

function _checkRewindTopUpQueueRole() internal view {
    _checkRole(REWIND_TOP_UP_QUEUE_ROLE);
}
```

Client's comments

Wrapper methods were added to save bytecode size. Adding similar wrappers for one-time-use combinations will defeat the purpose.

INFORMATIONAL-05	Minor gas optimizations	Fixed at: e57c3a1

Description

Lines:

- DepositQueueLib.sol#L10
- DepositQueueLib.sol#L37-L38
- DepositQueueOps.sol#L56-L57
- DepositQueueOps.sol#L320
- TopUpQueueOps.sol#L78-L81
- ParametersRegistry.sol#L629-L637
- ParametersRegistry.sol#L667-L670
- ParametersRegistry.sol#L689-L697
- SigningKeys.sol#L11
- UnstructuredStorage.sol#L10

1. In the **DepositQueueLib Batch.next()** function, the lower 128 bits are extracted via two shifts (**shl** + **shr**), while the same result can be achieved with a single **and** operation, which may be considered more semantically explicit.
2. In the **DepositQueueOps._loadAndAccountDeposits()** function, the subtraction **no.depositableValidatorsCount - keysCount** can be placed in an unchecked block, as underflow is not possible by design. The value of **keysCount** is derived from **Math.min(no.depositableValidatorsCount, ...)** earlier in the **DepositQueueOps.obtainDepositData()** function.
3. In the **TopUpQueueOps._allocateDeposits()**, the condition at TopUpQueueOps#L78 checks only **maxDepositAmount > 0** before computing the allocation. When the **limit** is zero, the block performs a redundant **Math.min(0, maxDepositAmount)** and no-op subtraction.
4. In **ParametersRegistry**, the **MarkedUint248** per-curve getter functions use inconsistent data location. The **_getKeyRemovalCharge()**, **_getGeneralDelayedPenaltyAdditionalFine()**, **_getKeysLimit()**, and **_getBadPerformancePenalty()** read via a **storage** reference, while **_getExitDelayFee()** and **_getMaxElWithdrawalRequestFee()** copy to **memory**. This is primarily a consistency issue with a minor gas overhead.
5. Multiple assembly blocks in **DepositQueueLib**, **SigningKeys**, and **UnstructuredStorage** lack the **"memory-safe"** annotation. In contrast, equivalent assembly in **TopUpQueueLib**, **SSZ**, **GIndex**, and other libraries already uses **assembly ("memory-safe")**. These blocks only operate on stack variables or storage slots and do not touch free memory, so they qualify as memory-safe. Without the annotation, the compiler cannot apply stack-to-memory optimizations via the Yul optimizer, potentially producing less efficient bytecode.

Recommendation

We recommend the following changes:

1. Define a named constant and replace the double shift with a single bitmask in **Batch.next()**:

```
uint256 constant NEXT_MASK = type(uint128).max;
n := and(self, NEXT_MASK)
```

2. Wrap the subtraction in **DepositQueueOps._loadAndAccountDeposits()** in an unchecked block:

```
uint32 newCount;
unchecked {
    newCount = no.depositableValidatorsCount - keysCount;
}
```

3. Extend the condition in **TopUpQueueOps._allocateDeposits()** to short-circuit when the limit is zero:

```
if (maxDepositAmount > 0 && limit > 0) {...}
```

4. We recommend choosing one format for this getter pattern and applying it consistently across all similar helpers, either **storage**-based reads or **memory**-based reads.

respect the free memory pointer.

INFORMATIONAL-06	Charge penalty recipient setter does not reject stETH address	Fixed at: cbe2c65

Description

Line: Accounting.sol#L602

Accounting._setChargePenaltyRecipient() validates a non-zero address but does not reject **LIDO** address explicitly. In this configuration, charging via **LIDO.transferShares(recipient, ...)** can attempt a transfer to the **LIDO** token contract itself, which will revert and break the charge flow.

Recommendation

We recommend adding an explicit **recipient != address(LIDO)** guard to prevent misconfiguration.

INFORMATIONAL-05	Minor gas optimizations	Fixed at: 24dfde2

Description

Lines:

- Accounting.sol#L479-L480
- Accounting.sol#L482
- Accounting.sol#L495
- FeeSplits.sol#L90
- BondLock.sol#L92-L99
- ValidatorStrikes.sol#L116
- Ejector.sol#L94
- WithdrawnValidatorLib.sol#L73
- WithdrawnValidatorLib.sol#L132

1. **Accounting._pullAndSplitFeeRewards()** already bounds **splittableShares** with **min(claimableShares, pendingToSplit)**, but **FeeSplits._decreasePendingSharesToSplit()** repeats bounding via **shares = shares > current ? current : shares**.
2. **Accounting._pullAndSplitFeeRewards()** enters the split settlement block when **hasSplits && claimableShares != 0**, even if **pendingToSplit == 0**. In this case, the flow still performs extra reads/calls and exits with zero effects.
3. When **amount == locked** (full unlock), the current implementation still reads **_getBondLockStorage().bondLock[nodeOperatorId].until** and passes it to **BondLock._changeBondLock()**. Since **BondLock._changeBondLock()** deletes the storage slot whenever the new amount is 0, the **.until** value is never used in this path.
4. **Ejector.ejectBadPerformer()** reverts with **IEjector.ZeroRefundRecipient()** when **refundRecipient** is zero. The only on-chain caller allowed by **onlyStrikes** is **ValidatorStrikes**, which already sets **refundRecipient = refundRecipient == address(0) ? msg.sender : refundRecipient** in **ValidatorStrikes.processBadPerformanceProof()**. So **refundRecipient** is not zero at the **Ejector** boundary.
5. For each validator in a batch, **WithdrawnValidatorLib._process()** -> **WithdrawnValidatorLib._fulfillExitObligations()** calls **IBaseModule(address(this)).ACCOUNTING()** and **EXIT_PENALTIES()** again. Those addresses are fixed for the module throughout the transaction, so each iteration repeats the same two lookups. Resolving each handle once before the loop (and threading **IAccounting** / **IExitPenalties** into **WithdrawnValidatorLib._process()** / **WithdrawnValidatorLib._fulfillExitObligations()**) removes redundant external calls without changing behavior.

Recommendation

We recommend the following changes:

1. Remove the duplicate clamp from **FeeSplits._decreasePendingSharesToSplit()**.
2. Add a direct **pendingToSplit != 0** guard in **Accounting._pullAndSplitFeeRewards()** before entering split settlement logic.

```
- if (hasSplits && claimableShares != 0) {...}

+ uint256 pendingToSplit = FeeSplits.getPendingSharesToSplit(nodeOperatorId);
+ if (hasSplits && claimableShares != 0 && pendingToSplit != 0) {...}
```

3. Short-circuit the full-unlock case:

```
function _unlock(uint256 nodeOperatorId, uint256 amount) internal {
    if (amount == 0) revert InvalidBondLockAmount();
    uint256 locked = getLockedBond(nodeOperatorId);
    if (locked < amount) revert InvalidBondLockAmount();
    if (locked == amount) {
        _changeBondLock(nodeOperatorId, 0, 0);
        return;
    }
    unchecked {
        _changeBondLock(nodeOperatorId, locked - amount, _getBondLockStorage().bondLock[nodeOperatorId].until);
    }
}
```

4. Remove the redundant **refundRecipient == address(0)** check from **Ejector.ejectBadPerformer()** and remove **ZeroRefundRecipient()** from **Ejector**.
5. Resolve **ACCOUNTING** and **EXIT_PENALTIES** once before the batch loop and pass the resulting contract handles to **WithdrawnValidatorLib._process()** / **WithdrawnValidatorLib._fulfillExitObligations()**, so the code does not repeat the two **IBaseModule(address(this))** getter calls on the module for every processed validator.

Client's comments

A different implementation from the proposed one was used for the **pendingToSplit == 0** case.
For the **WithdrawnValidatorLib._process()** changes, the tests showed a slight increase in gas usage for single-entry calls and insignificant savings for multiple items. The current version is a bit cleaner, so we decided to keep it as is.

INFORMATIONAL-08	Duplicate key pointer encoding logic in WithdrawnValidatorLib	Fixed at: cbe2c65

Description

Lines: WithdrawnValidatorLib.sol#L158-L162
WithdrawnValidatorLib._keyPointer() re-implements the same **(nodeOperatorId << 128) | keyIndex** encoding that already exists in KeyPointerLib.keyPointer(), resulting in duplicated pointer derivation logic across the codebase.
This duplication may increase long-term maintenance risk, as future changes to one implementation may not be mirrored in the other.

Recommendation

We recommend using **KeyPointerLib.keyPointer(uint256,uint256)** as the single source of truth and removing the local **_keyPointer()** implementation from **WithdrawnValidatorLib**.

INFORMATIONAL-09	Unused custom errors and a redundant inline interface	Fixed at: cbe2c65

Description

Lines:

- IBaseModule.sol#L134
- ICSModule.sol#L19-L22
- ICSModule.sol#L26
- BondCurvesLib.sol#L8-L13
- BondCurvesLib.sol#L134-L147
- DepositAllocatorGreedy.sol#L29-L30

Several custom errors are declared but never reverted, and are not referenced anywhere.
BondCurvesLib.sol additionally declares an internal **interface IBondCurves** re-declaring **InvalidBondCurveLength** / **InvalidBondCurveValues**, which already exist on the public **IBondCurve** interface the library imports. The two unused entries inside it (**InvalidBondCurveId**, **InvalidInitializationCurveId**) are never reverted, and the reverts in the library go through **IBondCurves** instead of **IBondCurve** for no functional reason.

Recommendation

We recommend removing the unused error declarations and, in **BondCurvesLib.sol**, deleting the inline **interface IBondCurves** and switching the five reverts to **IBondCurve.InvalidBondCurveLength()** / **IBondCurve.InvalidBondCurveValues()**.

INFORMATIONAL-10	Missing input validation enables key-pointer collision in view return values	Fixed at: d1b056e

Description

Lines:

- BaseModule.sol#L449-L451
- BaseModule.sol#L454-L456

KeyPointerLib implements the packed pointer as:

```
function keyPointer(uint256 nodeOperatorId, uint256 keyIndex) internal pure returns (uint256) {
    return (nodeOperatorId << 128) | keyIndex;
}
```

The layout is only injective if both **nodeOperatorId** and **keyIndex** fit in 128 bits. The module's external views do not enforce that:

```
function isValidatorSlashed(uint256 nodeOperatorId, uint256 keyIndex) external view returns (bool) {
    return _baseStorage().isValidatorSlashed[KeyPointerLib.keyPointer(nodeOperatorId, keyIndex)];
}

function isValidatorWithdrawn(uint256 nodeOperatorId, uint256 keyIndex) external view returns (bool) {
    return _baseStorage().isValidatorWithdrawn[KeyPointerLib.keyPointer(nodeOperatorId, keyIndex)];
}
```

If **nodeOperatorId** or **keyIndex** is larger than 128 bits, different input pairs can map to the same packed pointer. For example, **isValidatorSlashed(2**128, 5)** and **isValidatorSlashed(0, 5)** both map to pointer 5 and return the same value. The same issue applies to large **keyIndex** values, which can overlap with entries that use a non-zero **nodeOperatorId**.

Recommendation

We recommend validating **nodeOperatorId** and **keyIndex** ranges in both view functions before passing them to **KeyPointerLib.keyPointer()**, to prevent collisions from out-of-range inputs.

INFORMATIONAL-11	CuratedDepositAllocator._computeAllocations() precondition n > 0 is not enforced by CuratedDepositAllocator.allocateInitialDeposits()	Fixed at: 24dfde2

Description

Lines:

- CuratedDepositAllocator.sol#L54-L56
- CuratedDepositAllocator.sol#L434
- DepositAllocatorGreedy.sol#L32-L37

CuratedDepositAllocator._computeAllocations() documents that callers guarantee **allocationAmount > 0**, **n > 0**, and **step > 0**, and **DepositAllocatorGreedy._allocate()** repeats **state.sharesX96.length > 0** as an input invariant. **CuratedDepositAllocator.allocateInitialDeposits()** only enforces **depositsCount > 0**; if **CuratedDepositAllocator._collectDepositableOperatorsData()** returns no eligible operators (every operator has zero capacity or zero weight), **data.count == 0** and **data.alloc.sharesX96.length == 0** after **CuratedDepositAllocator._truncateDepositable()**, yet execution proceeds into **CuratedDepositAllocator._computeAllocations()**.

The current path is safe: **CuratedDepositAllocator._normalizeWeightsToShares()** iterates over a zero-length array (skipping the **Math.mulDiv()** that would otherwise divide by **data.weightSum == 0**) and **DepositAllocatorGreedy._allocate()** returns empty **allocations**. But the invariant documented on both helpers is violated.

Recommendation

We recommend either early-returning from **CuratedDepositAllocator.allocateInitialDeposits()** when **data.count == 0** (mirroring the existing **depositsCount == 0** guard), or amending the docstrings on **CuratedDepositAllocator._computeAllocations()** and **DepositAllocatorGreedy._allocate()** to make the empty-input path explicit.

Client's comments

> The docstring was updated.

INFORMATIONAL-12	BaseModule.updateDepositInfo() and BaseModule.updateDepositableValidatorsCount() lack an operator-existence check	Acknowledged

Description

Lines:

- BaseModule.sol#L382-L384
- BaseModule.sol#L261-L263
- CuratedModule.sol#L229-L232

BaseModule.updateDepositInfo() and **BaseModule.updateDepositableValidatorsCount()** are both callable by anyone and forward directly to their internal helpers without first verifying **nodeOperatorId < nodeOperatorsCount**. For a non-existent id, the calls are currently safe but still waste gas:

- **BaseModule.updateDepositInfo()** calls **CuratedModule._updateDepositInfo()**, which makes calls to **MetaRegistry.refreshOperatorWeight()** and **NodeOperatorOps.calculateDepositableValidatorsCount()** before reaching **BaseModule._applyDepositableValidatorsCount()**.
  In the end, nothing changes because the old and new values are both **0**, so the function exits early and the nonce is not incremented.
- **BaseModule.updateDepositableValidatorsCount()** behaves similarly. It still goes through **NodeOperatorOps.calculateDepositableValidatorsCount()**, but then exits early because the value remains **0**.

Recommendation

We recommend gating both external entry points with **BaseModule._onlyExistingNodeOperator(nodeOperatorId)**.

Client's comments

The operation for a non-existent operator is virtually a no-op, so we decided to keep it as is since we are tight on bytecode size anyway.

INFORMATIONAL-13	.length is re-evaluated on every iteration in for-loops	Fixed at: 24dfde2

Description

Lines:

- Accounting.sol#L485
- BaseModule.sol#L289
- BondCurvesLib.sol#L121
- BondCurvesLib.sol#L140
- CuratedDepositAllocator.sol#L130
- CuratedDepositAllocator.sol#L230
- CuratedDepositAllocator.sol#L404
- CuratedDepositAllocator.sol#L409
- CuratedDepositAllocator.sol#L467
- CuratedModule.sol#L59
- CuratedModule.sol#L275
- Ejector.sol#L57
- Ejector.sol#L133
- FeeSplits.sol#L50
- MetaRegistry.sol#L268
- MetaRegistry.sol#L276
- MetaRegistry.sol#L290
- MetaRegistry.sol#L316
- MetaRegistry.sol#L413
- ParametersRegistry.sol#L273
- ParametersRegistry.sol#L287
- ParametersRegistry.sol#L741
- StakeTracker.sol#L54
- ValidatorStrikes.sol#L110
- ValidatorStrikes.sol#L137
- ValidatorStrikes.sol#L163
- WithdrawnValidatorLib.sol#L33

The for-loops listed above all re-evaluate **arr.length** on every iteration instead of caching it once before the loop, despite the body never mutating the array. The cost per iteration depends on the array's data location.

Storage-array loops in **MetaRegistry** — **MetaRegistry._resetGroup()** iterates **group.subNodeOperatorIds** and **group.externalOperators** (both **storage**); **MetaRegistry._totalExternalStake()** takes **externalOperators** as a **storage** parameter. Each iteration re-issues an **SLOAD** on the length slot (~2100 gas cold, ~100 warm), which dominates the per-iteration cost as group size grows.

Recommendation

We recommend caching the length into a local before entering each loop:

```
uint256 n = arr.length;
for (uint256 i; i < n; ++i) { ... }
```

We recommend prioritising the **MetaRegistry** storage loops (**_resetGroup()**, **_totalExternalStake()**), then the deposit hot path (**obtainDepositData()**, **_validateTopUpPublicKeys()**, the **CuratedDepositAllocator.*** family), then the remaining entries.

Client's comments

> According to our tests, extracting the array length outside the loop is reasonable for some storage and nested calldata arrays. For memory arrays, it's either already well-optimized by the compiler or the gas costs for memory reads are negligible.

INFORMATIONAL-14	ExitPenalties.processExitDelayReport() and ExitPenalties.processTriggeredExit() do not verify pubkey & node operator binding	Acknowledged

Description

Lines:

- ExitPenalties.sol#L49-L53
- ExitPenalties.sol#L68-L73
- BaseModule.sol#L357-L379

ExitPenalties.processExitDelayReport(nodeOperatorId, publicKey, ...) and **ExitPenalties.processTriggeredExit(nodeOperatorId, publicKey, ...)** accept **(nodeOperatorId, publicKey)** from the module – forwarded by **BaseModule.reportValidatorExitDelay()** and **BaseModule.onValidatorExitTriggered()**, both gated only by **STAKING_ROUTER_ROLE**. Neither the module nor **ExitPenalties** verifies that **publicKey** is actually a key registered for **nodeOperatorId**. Penalties are stored under **KeyPointerLib.keyPointer(nodeOperatorId, publicKey)**, while later consumption in **WithdrawnValidatorLib._process()** looks up the entry using **pubkey = SigningKeys.loadKeys(nodeOperatorId, keyIndex, 1)** – i.e. the operator's actual stored key.

If StakingRouter (or the upstream validators-exit-bus oracle) ever supplies a **(nold_A, pubkey_B)** mismatch through a bug or oracle misreport, the penalty silently lands in an unreachable storage slot, and the operator escapes the penalty when their real key is later withdrawn.

Recommendation

We recommend verifying that **publicKey** belongs to **nodeOperatorId** before recording the penalty.

Client's comments

> We treat the StakingRouter as a trusted actor and estimate the probability of delivering an incorrect public key as low. The sanity checks that are possible with the provided input are implemented.

INFORMATIONAL-15	Missing bond curve existence validation causes unnecessary global refreshes	Fixed at: 24dfde2

Description

Lines: MetaRegistry.sol#L133-L141

MetaRegistry.setBondCurveWeight() writes **bondCurveWeight[curveId]** without validating that **curveId** exists in Accounting, and then unconditionally calls **MODULE.requestFullDepositInfoUpdate()**. As a result, an account with **SET_BOND_CURVE_WEIGHT_ROLE** can assign a weight to a non-existent curve while still triggering a full operator refresh. Since no operator can use a non-existent curve, this full deposit info update is unnecessary and adds avoidable overhead.

Recommendation

We recommend validating **curveId** existence before storing a weight, for example, via an explicit existence check against Accounting curve storage.

```
function setBondCurveWeight(uint256 curveId, uint256 weight) external onlyRole(SET_BOND_CURVE_WEIGHT_ROLE) {
    MetaRegistryStorage storage $ = _storage();
+     if (curveId >= ACCOUNTING.getCurvesCount()) revert IBondCurve.InvalidBondCurveId();
    if (weight != 0 && weight < MAX_BP) revert InvalidBondCurveWeight();
    if ($.bondCurveWeight[curveId] == weight) revert SameBondCurveWeight();

    $.bondCurveWeight[curveId] = weight;
    emit BondCurveWeightSet(curveId, weight);
    MODULE.requestFullDepositInfoUpdate();
  }
```

INFORMATIONAL-16	Grouped operator can be configured with zero share	Acknowledged

Description

Lines: MetaRegistry.sol#L284-L310

MetaRegistry._storeSubOperators() allows adding a sub-operator with **share = 0** while still assigning it to a group. This configuration has no practical allocation purpose: operators with zero share are effectively non-participating.

Recommendation

We recommend rejecting sub-operators with **share == 0** during group creation/update.

Client's comments

The behaviour is intentional. A group with 0 share sub node operator can be created to force node operator to create sub node operator of specific type in the module. It will allow to change the target distribution within the node operator should the requirements change.

INFORMATIONAL-17	Inconsistent top-up eligibility and quantization can reduce effective distribution	Fixed at: c607ba5

Description

Lines:

- CuratedDepositAllocator.sol#L273-L287
- DepositAllocatorGreedy.sol#L57-L63
- DepositAllocatorGreedy.sol#L73-L77
- CuratedDepositAllocator.sol#L418-L423

Top-up flow uses different eligibility thresholds across stages. During baseline collection, operators are included in **weightSum** and **totalCurrent** when raw **capacity > 0** and weight is non-zero, but during greedy allocation, capacity is quantized to **TOP_UP_STEP**, making sub-step capacity unusable (**0 < capacity < TOP_UP_STEP**).
Operator-level allocations are also computed before per-key **topUpLimit** caps are enforced, so an operator may still affect baseline totals while being effectively non-allocatable at distribution, due to **topUpLimit = 0**.
This creates a mismatch between contributors to **weightSum** / **totalCurrent** and operators that can actually receive top-ups. As a result, effective share calculation may be diluted, reducing distribution efficiency and leaving part of a round's funds undistributed.

Recommendation

We recommend excluding effectively non-allocatable operators from baseline **weightSum** and **totalCurrent** calculations (or applying the same quantization/limit constraints before baseline aggregation).

Client's comments

Sub-step capacity operators are excluded from the baseline. It's not trivial to take into account per-key top-up limits, but we assume the balances will eventually be updated in the StakeTracker and a node operator with sub-step capacity across their keys will be excluded from the baseline.

INFORMATIONAL-18	CuratedModule._updateDepositInfo() duplicates the BaseModule implementation instead of calling super	Fixed at: 24dfde2

Description

Lines:

- CuratedModule.sol#L231
- BaseModule.sol#L639-L641

The **CuratedModule._updateDepositInfo()** function calls **MetaRegistry.refreshOperatorWeight()** and then invokes **_updateDepositableValidatorsCount()** directly instead of delegating the second step to **super._updateDepositInfo()**.
Currently, **BaseModule._updateDepositInfo()** only forwards to **_updateDepositableValidatorsCount()** with **incrementNonceIfUpdated: true**, so behavior matches the base implementation; the concern is maintainability and avoiding silent divergence if the base hook gains further responsibilities.

Recommendation

We recommend replacing the direct **_updateDepositableValidatorsCount({ nodeOperatorId: nodeOperatorId, incrementNonceIfUpdated: true })** call as following:

```
function _updateDepositInfo(uint256 nodeOperatorId) internal override {
    _metaRegistry().refreshOperatorWeight(nodeOperatorId);
    super._updateDepositInfo(nodeOperatorId);
}
```

INFORMATIONAL-19	**BondCurve.getBondAmountByKeysCount() and BondCurve.getKeysCountByBondAmount() revert with a generic panic for invalid curveId**	**Fixed at: d8de8bc**

Description

Lines:

- BondCurve.sol#L67-L73
- BondCurvesLib.sol#L46-L52
- BondCurvesLib.sol#L70-L78

BondCurve.getBondAmountByKeysCount() and **BondCurve.getKeysCountByBondAmount()** forward **curveId** into **BondCurvesLib**, where the library indexes **bondCurveStorage.bondCurves[curveId]** without first checking that the curve exists. For an invalid **curveId**, the call can revert with Solidity's generic array out-of-bounds panic instead of **IBondCurve.InvalidBondCurveId**.

Other curve-facing methods, such as **BondCurve._setBondCurve()** and **BondCurve.getCurveInfo()**, already validate **curveId** and use the custom error. The impact is limited to view/calculation entry points, but the public API is inconsistent for callers and off-chain tooling.

Recommendation

We recommend validating **curveId** in **BondCurvesLib.getBondAmountByKeysCount()** and **BondCurvesLib.getKeysCountByBondAmount()** before indexing **bondCurveStorage.bondCurves[curveId]**, and reverting with **IBondCurve.InvalidBondCurveId()** when the curve does not exist.

INFORMATIONAL-20	Gas Optimizations	Fixed at: 24dfde2

Description

Lines:

- DepositAllocatorGreedy#L39-L71
- CuratedDepositAllocator#L183
- WithdrawnValidatorLib.sol#L149
- MetaRegistry.sol#L109-L115
- MetaRegistry.sol#L339-L342

1. **DepositAllocatorGreedy._allocate()** calls **_quantize(remainder, step)** on each loop iteration. Since **remainder** only decreases by **toGive**, pre-quantizing once and tracking **quantizedRemainder** removes repeated quantization work and allows dropping the **toGive == 0** branch.
2. **CuratedDepositAllocator._collectDepositableOperatorsData()** uses **if (weight == 0) continue** after **capacity > 0** check. Currently, **no.depositableValidatorsCount** is non-zero only when **weight > 0**, so **capacity > 0** already guarantees **weight > 0**.
3. The inner **Math.max(MIN_ACTIVATION_BALANCE, ...)** in **WithdrawnValidatorLib._getPenaltyMultiplier()** is dead code: the caller already passes **max(minExpectedBalance, exitBalance)** with **minExpectedBalance ≥ MIN_ACTIVATION_BALANCE**.
4. **MetaRegistry.setOperatorMetadataAsOwner()** reverts when **ownerEditsRestricted** is true, then always writes false back into storage via **MetaRegistry._storeOperatorMetadata()** - a provably same value SSTORE (~100 gas) plus wasted loads. That redundant work runs on every owner name/description edit.
5. In **MetaRegistry._refreshOperatorWeight()**, when an operator's effective weight changes, the group's cumulative weight sum is updated:

```
if (oldWeight != newWeight) {
   $.effectiveWeightCache.groupEffectiveWeightSum[groupId] =
      $.effectiveWeightCache.groupEffectiveWeightSum[groupId] +
      newWeight -
      oldWeight;
}
```

The expression **groupEffectiveWeightSum + newWeight – oldWeight** is evaluated left-to-right as **(groupEffectiveWeightSum + newWeight) – oldWeight**. Both operations are safe due to protocol invariants:

- Underflow is impossible. The **oldWeight** value is always part of **groupEffectiveWeightSum**, so **groupEffectiveWeightSum >= oldWeight** holds by invariant.
- Overflow is impossible in practice. Effective weights are derived from **bondCurveWeight** and **share** (uint16, capped at 10000), producing values negligible relative to **type(uint256).max**.

Recommendation

We recommend the following changes:

1. Refactor **DepositAllocatorGreedy._allocate()** to pre-quantize **allocationAmount** into **quantizedRemainder** before the loop, use **quantizedRemainder** directly in **toGive**, and compute **allocated** against the quantized value.

```
function _allocate(
      AllocationState memory state,
      uint256 allocationAmount,
      uint256 step
) internal pure returns (uint256 allocated, uint256[] memory allocations) {
       uint256 n = state.sharesX96.length;
       allocations = new uint256[](n);

       PackedSortKey[] memory heap = _maxHeapKeysByImbalanceDesc(state, allocationAmount, step);
       uint256 heapSize = heap.length;
-      uint256 remainder = allocationAmount;
+      uint256 quantizedRemainder = _quantize(allocationAmount, step);
+      allocationAmount = quantizedRemainder;

-      while (heapSize > 0 && remainder > 0) {
+      while (heapSize > 0 && quantizedRemainder > 0) {
          PackedSortKey key = heap.popMax(heapSize);
          unchecked {
             --heapSize;
          }
          uint256 opIdx = key.unpackIndex();
          uint256 possible = Math.min(key.unpackImbalance(), _quantize(state.capacities[opIdx], step));

          if (possible == 0) continue;
-         uint256 toGive = Math.min(possible, _quantize(remainder, step));
+         uint256 toGive = Math.min(possible, quantizedRemainder);

-         if (toGive == 0) break;
          allocations[opIdx] = toGive;
          unchecked {
-             remainder -= toGive;
+             quantizedRemainder -= toGive;
          }
      }
-     allocated = allocationAmount - remainder;
+     allocated = allocationAmount - quantizedRemainder;
  }
```

2. Remove **if (weight == 0) continue** in the **CuratedDepositAllocator._collectDepositableOperatorsData()** loop.
3. Remove the redundant **Math.max**, keeping only the **Math.min(MAX_EFFECTIVE_BALANCE, ...)** clamp and update the existing unit tests that call **WithdrawnValidatorLib._getPenaltyMultiplier()** with balances below 32 ETH.

```
function _getPenaltyMultiplier(uint256 balance) internal pure returns (uint256 penaltyMultiplier) {
-       balance = Math.max(ValidatorBalanceLimits.MIN_ACTIVATION_BALANCE, balance);
        balance = Math.min(ValidatorBalanceLimits.MAX_EFFECTIVE_BALANCE, balance);
        penaltyMultiplier = balance / PENALTY_QUOTIENT; // 10**18 / 10**18 = 1
}
```

4. Pass false directly instead of forwarding **ownerEditsRestricted**, saving one warm SLOAD (~100 gas) per call.
5. Wrap the arithmetic in an unchecked block:

```
 if (oldWeight != newWeight) {
+    unchecked {
       $.effectiveWeightCache.groupEffectiveWeightSum[groupId] =
         $.effectiveWeightCache.groupEffectiveWeightSum[groupId] +
         newWeight -
         oldWeight;
+    }
 }
```

Client's comments

A slightly different implementation was used to follow the recommendation for **DepositAllocatorGreedy._allocate()**.
We kept the **if (weight == 0) continue** branch in **CuratedDepositAllocator._collectDepositableOperatorsData()** given the insignificant gas costs, to keep the method more sound without the need to document the invariant.
The operator metadata change method is supposed to be called pretty infrequently. In favour of a cleaner implementation, it was decided to keep it as is.
WithdrawnValidatorLib._getPenaltyMultiplier was changed to accept the clamped balance value.

STATE
MIND